

Introduction to Systems Programming

Antonio Carzaniga

Faculty of Informatics
Università della Svizzera italiana

September 14, 2026

- on iCorsi: ***SysProg2026F***

- ***<https://www.inf.usi.ch/carzaniga/edu/sysprog/>***

- on iCorsi: ***SysProg2026F***

- ***<https://www.inf.usi.ch/carzaniga/edu/sysprog/>***

- Announcements

 - ▶ through iCorsi

You are responsible for reading the announcements (sent through iCorsi)

- on iCorsi: ***SysProg2026F***

- ***<https://www.inf.usi.ch/carzaniga/edu/sysprog/>***

- Announcements

- ▶ through iCorsi

You are responsible for reading the announcements (sent through iCorsi)

- Office hours

- ▶ Antonio Carzaniga: *by appointment*

- ▶ Pouria Peykani Sani: *by appointment*

- ▶ Pasquale Polverino: *by appointment*

- Schedule with *proposed changes*

■ Schedule with *proposed changes*

- ▶ Monday lectures: 9:00–10:40, with a 10-minute break
- ▶ Wednesday lectures: 9:00–10:40, with a 10-minute break

■ Schedule with *proposed changes*

- ▶ Monday lectures: 9:00–10:40, with a 10-minute break
- ▶ Wednesday lectures: 9:00–10:40, with a 10-minute break

■ Exercise sessions

- ▶ Wednesday 15:30–17:00 in D0.10

■ Schedule with *proposed changes*

- ▶ Monday lectures: 9:00–10:40, with a 10-minute break
- ▶ Wednesday lectures: 9:00–10:40, with a 10-minute break

■ Exercise sessions

- ▶ Wednesday 15:30–17:00 in D0.10

■ Midterm exam: mark your calendars!

- ▶ Monday 2 November 9:00–11:00

- Focus: *concrete and practical* systems programming
 - ▶ still with good software engineering practices

- Focus: *concrete and practical* systems programming
 - ▶ still with good software engineering practices
- Structure: *reading/lecture* + *in-class exercises* + *homework*

- Focus: ***concrete and practical*** systems programming
 - ▶ still with good software engineering practices
- Structure: ***reading/lecture*** + ***in-class exercises*** + ***homework***
- *Lectures*
 - ▶ *preliminary reading* plus *interactive lectures*
 - ▶ in-class exercises
 - ▶ so, you should have your computer handy (and charged)

- Focus: ***concrete and practical*** systems programming
 - ▶ still with good software engineering practices
- Structure: ***reading/lecture*** + ***in-class exercises*** + ***homework***
- *Lectures*
 - ▶ *preliminary reading* plus *interactive lectures*
 - ▶ in-class exercises
 - ▶ so, you should have your computer handy (and charged)
- *Homework*
 - ▶ a programming assignment *every week*
 - ▶ a few assignments may be graded (we'll tell you which ones)
 - ▶ all assignments will be discussed in class

- +10% homework: programming assignments
 - ▶ grades added together, thus resulting in a weighted average
- +30% midterm exam
 - ▶ in-class, concrete programming exercise (using a computer)
- +60% final exam
 - ▶ in-class, concrete programming exercise (using a computer)
- $\pm 10\%$ instructor's discretionary evaluation
 - ▶ participation
 - ▶ extra credits
 - ▶ trajectory
 - ▶ ...

- +10% homework: programming assignments
 - ▶ grades added together, thus resulting in a weighted average
- +30% midterm exam
 - ▶ in-class, concrete programming exercise (using a computer)
- +60% final exam
 - ▶ in-class, concrete programming exercise (using a computer)
- $\pm 10\%$ instructor's discretionary evaluation
 - ▶ participation
 - ▶ extra credits
 - ▶ trajectory
 - ▶ ...
- -100% plagiarism/cheating penalties

Deadlines are firm.

Deadlines are firm.

- You know what I mean...
- Three-days-and-you're-out rule applies...

Plagiarism and Cheating

Plagiarism and Cheating

A student must never take material, from any source, and present it as his or her own. Doing so means committing plagiarism.

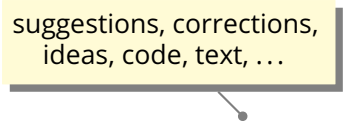
Plagiarism and Cheating

A student must never take material, from any source, and present it as his or her own. Doing so means committing plagiarism.

A student must abide by explicit as well as implicit instructions and regulations on exams and assignments.

Plagiarism and Cheating

suggestions, corrections,
ideas, code, text, ...



A student must never take material, from any source, and present it as his or her own. Doing so means committing plagiarism.

A student must abide by explicit as well as implicit instructions and regulations on exams and assignments.

Plagiarism and Cheating

suggestions, corrections,
ideas, code, text, ...

people, books, course notes,
stackoverflow, CheatGPT, ...

A student must never take material, from any source, and present it as his or her own. Doing so means committing plagiarism.

A student must abide by explicit as well as implicit instructions and regulations on exams and assignments.

Plagiarism and Cheating

suggestions, corrections,
ideas, code, text, ...

people, books, course notes,
stackoverflow, CheatGPT, ...

A student must never take material, from any source, and present it as his or her own. Doing so means committing plagiarism.

A student must abide by explicit as well as implicit instructions and regulations on exams and assignments.

- **Penalties:** plagiarism and cheating on an assignment or an exam will result in
 - ▶ failing that assignment or that exam
 - ▶ losing one or more points *in the final note!*
- Plagiarism and cheating cases will be reported to the Dean's office
- Penalties may be escalated...

What's the Point?

What's the Point?

You are here ***to learn!***

What's the Point?

You are here ***to learn!***

- Understanding programs
- Solving problems

How does one learn?

How does one learn?

Practice, practice, practice!

How does one learn?

Practice, practice, practice!

- ***You*** have to do it
- Feedback (e.g., automated tests)
- Expert guidance

How to Learn Systems Programming

How to Learn Systems Programming

1. Solve a programming problem

How to Learn Systems Programming

1. Solve a programming problem

- ▶ read the problem specification carefully
- ▶ develop a high-level strategy
- ▶ implement and test your strategy

take your time!

effortful, “slow thinking”
methodical practice

How to Learn Systems Programming

1. Solve a programming problem

- ▶ read the problem specification carefully
- ▶ develop a high-level strategy
- ▶ implement and test your strategy

take your time!

effortful, “slow thinking”
methodical practice

2. If you are stuck, ask somebody to help you out (your teacher is always happy to help you!)

...But *do not simply copy code!*

How to Learn Systems Programming

1. Solve a programming problem

take your time!

- ▶ read the problem specification carefully
- ▶ develop a high-level strategy
- ▶ implement and test your strategy

effortful, “slow thinking”
methodical practice

2. If you are stuck, ask somebody to help you out (your teacher is always happy to help you!)

...But *do not simply copy code!*

3. When you're done—when your own solution is *complete*—analyze other solutions, such as the solutions presented in class

How to Learn Systems Programming

1. Solve a programming problem

take your time!

- ▶ read the problem specification carefully
- ▶ develop a high-level strategy
- ▶ implement and test your strategy

effortful, “slow thinking”
methodical practice

2. If you are stuck, ask somebody to help you out (your teacher is always happy to help you!)

...But *do not simply copy code!*

3. When you're done—when your own solution is *complete*—analyze other solutions, such as the solutions presented in class

4. Go to step 1

What about automation?

“AI”, LLMs, etc.

What about automation?

“AI”, LLMs, etc.

Remember, the point is for you ***to learn***

Would it work for learning in other areas?
(e.g., driving, music, sport, chess, ...)

What about automation?

“AI”, LLMs, etc.

Remember, the point is for you ***to learn***

Would it work for learning in other areas?
(e.g., driving, music, sport, chess, ...)

YOU *have to solve the exercises*

“The first principle is that you must not fool yourself
and you are the easiest person to fool.”

—Richard P. Feynman

*Now on to **Systems Programming!***

What is Systems Programming?

What is Systems Programming?

- Interfacing with a “system” (as opposed to a user)
 - ▶ rigid interfaces
 - ▶ complex interfaces

What is Systems Programming?

- Interfacing with a “system” (as opposed to a user)

- ▶ rigid interfaces
- ▶ complex interfaces

- Engineering for a non trivial platform

- ▶ non-trivial performance profiles
- ▶ going beyond algorithmic complexity

The Language(s) of Systems Programming

The Language(s) of Systems Programming

- Mostly C, ~~and a bit of C++~~

The Language(s) of Systems Programming

- Mostly C, ~~and a bit of C++~~
- A lot of software is written in C or C++
 - ▶ the vast majority of the programs running on your computer (now!)
 - ▶ including the operating system
 - ▶ a lot more *new* software will be written in C/C++
 - ▶ usually the most crucial components of large systems

The Language(s) of Systems Programming

- Mostly C, ~~and a bit of C++~~
- A lot of software is written in C or C++
 - ▶ the vast majority of the programs running on your computer (now!)
 - ▶ including the operating system
 - ▶ a lot more *new* software will be written in C/C++
 - ▶ usually the most crucial components of large systems
- Available on virtually every computer platform
 - ▶ from embedded controllers to supercomputers

The Language(s) of Systems Programming

- Mostly C, ~~and a bit of C++~~
- A lot of software is written in C or C++
 - ▶ the vast majority of the programs running on your computer (now!)
 - ▶ including the operating system
 - ▶ a lot more *new* software will be written in C/C++
 - ▶ usually the most crucial components of large systems
- Available on virtually every computer platform
 - ▶ from embedded controllers to supercomputers
- System programming
 - ▶ “low-level” programming (e.g., a device driver)
 - ▶ “high-level” programming (e.g., the Firefox web browser)

The Language(s) of Systems Programming

- Mostly C, ~~and a bit of C++~~
- A lot of software is written in C or C++
 - ▶ the vast majority of the programs running on your computer (now!)
 - ▶ including the operating system
 - ▶ a lot more *new* software will be written in C/C++
 - ▶ usually the most crucial components of large systems
- Available on virtually every computer platform
 - ▶ from embedded controllers to supercomputers
- System programming
 - ▶ “low-level” programming (e.g., a device driver)
 - ▶ “high-level” programming (e.g., the Firefox web browser)
- Relatively simple (C) but still powerful language
 - ▶ C++ is definitely not that simple
 - ▶ like any serious tool, C and C++ have hidden complexities...

Getting Started: One, Two, Three!

Getting Started: One, Two, Three!

1. Edit the program *ciao.c*

```
#include <stdio.h>

int main () {
    print("Ciao!\n");
}
```

Getting Started: One, Two, Three!

1. Edit the program *ciao.c*

```
#include <stdio.h>

int main () {
    print("Ciao!\n");
}
```

2. Compile the program (i.e., run the compiler)

```
$ cc ciao.c -o ciao
```

Getting Started: One, Two, Three!

1. Edit the program *ciao.c*

```
#include <stdio.h>

int main () {
    print("Ciao!\n");
}
```

2. Compile the program (i.e., run the compiler)

```
$ cc ciao.c -o ciao
```

3. Run the program

```
$ ./ciao
```

Getting Started with *Make*

1. Edit the program *ciao3.c*

```
#include <stdio.h>

int main() {
    printf ("I said Ciao already!\n");
}
```

Getting Started with *Make*

1. Edit the program *ciao3.c*

```
#include <stdio.h>

int main() {
    printf ("I said Ciao already!\n");
}
```

2. Compile the program using *make*

```
$ make ciao3
```

Getting Started with *Make*

1. Edit the program *ciao3.c*

```
#include <stdio.h>

int main() {
    printf ("I said Ciao already!\n");
}
```

2. Compile the program using *make*

```
$ make ciao3
```

3. Run the program

```
$ ./ciao3
```


Try compiling the program:

```
#include <stdio.h>

int main () {
    print("I said Ciao already!\n");
}
```

Try compiling the program:

```
#include <stdio.h>

int main () {
    print("I said Ciao already!\n");
}
```

You should get some errors:

```
errors.c: In function 'main':
errors.c:4:5: error: implicit declaration of function 'print'; did you mean
'printf'? . . .
    4 |     print("I said Ciao already!\n");
      |     ^~~~~
      |     printf
```

The function you will use to print data in C is printf:

```
#include <stdio.h>

int main() {
    printf("My name is %s.\nI was %d in the year 2000.\n",
        "Antonio", 2000 - 1969);
}
```

The function you will use to print data in C is printf:

```
#include <stdio.h>

int main() {
    printf("My name is %s.\nI was %d in the year 2000.\n",
        "Antonio", 2000 - 1969);
}
```

The first argument is a **format string** that includes **conversion specifications**, beginning with a % sign, that tell printf how to interpret its other arguments:

- %d prints an integer in decimal notation
- %c prints an integer as a character
- %g prints a float in decimal notation
- ... *see the documentation of printf()*

Minimal (One-Byte) I/O

- `getchar()` reads the next character (byte) from the “standard input”
 - ▶ returns an `int` value
 - ▶ returns EOF at the end of file

Minimal (One-Byte) I/O

- `getchar()` reads the next character (byte) from the “standard input”
 - ▶ returns an `int` value
 - ▶ returns EOF at the end of file

...***you should read the documentation!***

Minimal (One-Byte) I/O

- `getchar()` reads the next character (byte) from the “standard input”
 - ▶ returns an `int` value
 - ▶ returns EOF at the end of file

...***you should read the documentation!***
- **Exercise:** write a program that counts and prints the number of characters (bytes) in its standard input stream

Minimal (One-Byte) I/O

- `getchar()` reads the next character (byte) from the “standard input”
 - ▶ returns an `int` value
 - ▶ returns EOF at the end of file*...you should read the documentation!*
- **Exercise:** write a program that counts and prints the number of characters (bytes) in its standard input stream

```
#include <stdio.h>

int main () {
    int i = 0;
    while (getchar() != EOF)
        ++i;
    printf ("%d characters\n", i);
    return 0;
}
```

Examples...

Minimal (One-Byte) I/O

`putchar(int c)` writes one byte to the “standard output”

putchar(int c) writes one byte to the “standard output”

■ **Exercise:** what does this program do?

```
#include <stdio.h>
#include <limits.h>

int main() {
    int c;
    while ((c = getchar()) != EOF) {
        c += 3;
        if (c > UCHAR_MAX)
            c = UCHAR_MIN + (c - UCHAR_MAX);
        putchar(c);
    }
}
```

putchar(int c) writes one byte to the “standard output”

- **Exercise:** what does this program do?

```
#include <stdio.h>
#include <limits.h>

int main() {
    int c;
    while ((c = getchar()) != EOF) {
        c += 3;
        if (c > UCHAR_MAX)
            c = UCHAR_MIN + (c - UCHAR_MAX);
        putchar(c);
    }
}
```

- **Exercise:** write a program that inverts the transformation of the program above

Run this program:

```
#include <stdio.h>

int main () {
    putchar(67);
    putchar(105);
    putchar(97);
    putchar(111);
    putchar(33);
    putchar(10);
}
```

Now run this other program:

```
#include <stdio.h>

int main () {
    putchar(240);
    putchar(159);
    putchar(153);
    putchar(130);
    putchar(10);
}
```

C and C++ have the usual control structures:

- for
- while
- do...while
- switch
- if...else...
- break
- continue
- return

```
int f(int n) {  
    int p, pp, r;  
    switch(n) {  
        case 0:  
        case 1: return n;  
        default:  
            p = 1;  
            pp = 0;  
            do {  
                r = p + pp;  
                pp = p;  
                p = r;  
            } while (--n > 1);  
            return r;  
        }  
    }
```

Control Structures: Exercise 1

- Write a program called `diamond.c` that, given a number n , prints (on the terminal) an $n \times n$ diamond like this one (6×6):

#####

Control Structures: Exercise 2

- Rewrite without using the switch statement

```
int main () {  
    int c;  
    while ((c = getchar()) != EOF) {  
        switch (c) {  
            case ' ': putchar('\n'); break;  
            case '\n': putchar('\n'); putchar('\n'); break;  
            case 'a':  
            case 'e':  
            case 'i':  
            case 'o':  
            case 'u': putchar(c); putchar('s');  
            default: putchar(c);  
        }  
    }  
}
```

Control Structures: Exercise 3

- Write a program that *reverts* this input/output transformation:

```
int main () {  
    int c;  
    while ((c = getchar()) != EOF) {  
        switch (c) {  
            case ' ': putchar('\n'); break;  
            case '\n': putchar('\n'); putchar('\n'); break;  
            case 'a':  
            case 'e':  
            case 'i':  
            case 'o':  
            case 'u': putchar(c); putchar('s');  
            default: putchar(c);  
        }  
    }  
}
```

Homework Assignment: wordcount

- Write a program called *wordcount* that counts the words in the standard input. A *word* is a sequence of one or more characters delimited by white space.
 - ▶ the output should be the same as the command:

```
$ wc -w
```